

REPORT 2.3 — The recorder bake-off apps, reviewed at xhigh before the course (Fable, 2026-09-14)

Item: docs/handoff/FOR_FABLE.md 2.3. Tree reviewed: 645355a (clean, not pushed; 16 ahead of origin/main). Effort **xhigh** on his word, 2026-09-14: "x-high your are good to go". The apps on his phone were built from 44531f3; between that and 645355a the app code differs in two notification titles and the version (`git diff 44531f3..645355a -- android/bakeoff/app/src: 2 lines`), so this review covers both the build that recorded the round and the build that goes on the phone next.

His words that set the bar, 2026-09-13: "99% coverage, no gap > 20 s (Recommended)", asked as "What result does a recorder need to pass? Same coverage measure as the handoff's Section 9 table, over a round-length carry with the screen locked and a music app in use." And on this review: "let Fable analyze the data then we can talk about the round after it".

Tags: [measured] read from the data or the binary; [inferred] follows from measurements but was not itself measured; [design] an Android or SDK rule from its own documentation; [verify] I could not check it from here.

1. The answer, for the phone

May the two apps go on the course? Yes. Both APKs at 645355a are safe to carry and to play beside golf-tracker. Nothing in either app can lose or corrupt a row it was handed, neither feeds a round, and on the one real round so far each kept one fix a second for two hours with no gap over 2 s ($n = 1$ round, 7,237 and 7,256 fixes). Verdict detail in Section 3.

Does today's round count? Yes, as the on-course test. No, as the locked-screen test. It proves the sky, the pocket, the push cart, two hours, and 3.1 m accuracy for both ($n = 7,230$ shared fixes). It cannot satisfy your bar's clause "with the screen locked and a music app in use": the screen was off at 76 of 1,445 heartbeats (5.3%) and music played at 0. In the one screen-off stretch, 6.3 min, both apps held 1 Hz while golf-tracker got nothing ($n = 1$ stretch, 376 fixes each). Neither app has passed the bar yet; neither has failed it.

What the locked-screen test must be (details in Section 9):

1. Two carries, not one: **both apps together** (the comparison), then **GPS Transistor alone**. GPS Custom holds a CPU wake lock for its whole session, a wake lock keeps the CPU on for every app on the phone, and the transistorsoft SDK holds none of its own (read from its binary), so T has never yet run on its own power path.
2. Each carry at least 2 hours, walking outdoors for most of it, phone locked in a pocket, unplugged, no golf-tracker running (Chrome closed), a music app playing through headphones for at least half of it.
3. It counts as locked only if the heartbeats say so: screen off at 90% or more of beats, music at 50% or more. Unlocking to use the phone is fine; every unlock is logged. Do not open either test app until the end.
4. Started and stopped from the desktop as now; scored by `tools/track-coverage.py` as now, per app, START to STOP, edges included.

2. What was read and run

- Every source file under `android/bakeoff/app/src/` (both flavors, shared code, the test and its fixture), `build.gradle.kts`, both manifests, `bakeoff.ps1`, `tools/track-coverage.py`, `android/bakeoff/README.md`, `docs/HANDOFF-native-build.md` Section 9, the round's README and its committed outputs, `docs/DECISIONS_LOG.md` (the Opus entries), `docs/bakeoff-test-week.md`, `docs/handoff/NEXT_CHAT_2026-09-14.md`, `docs/CATCHUP-rev3-rev4.md`, the memory index.
- The round's four data files and T's SDK store, read only, with a script in my scratchpad (never in the repo).
- transistorsoft's `tslocationmanager-4.5.1.aar` from the Gradle cache, unpacked into my scratchpad: its manifest, and its 642 classes searched by string and read with `javap -c` where a question needed the bytecode.
- Fetched, not recalled: Android's list of exemptions from the background-start restriction on foreground services ([restrictions-bg-start](#)), and transistorsoft's GeolocationConfig page ([docs](#)).
- Run: the three commands in the round README; `--self-test`; both apps' JVM unit tests; a mutation of the Python gap rule; the tool over every export in `docs/roundDownloads/` (read only; nothing from there is staged).

Checks [measured]:

Check	Result
python tools/track-coverage.py docs/bakeoff-data/.../golf-bakeoff	byte-identical to the committed score-bakeoff.txt (BOM aside)
... golf-tracker-20260914-1807.json	identical to score-golf-tracker-export.txt
python .../compare-web-gaps.py	identical to web-gaps-vs-native.txt
--self-test	14/14
JVM unit tests, testHandwrittenDebugUnitTest / testTransistorDebugUnitTest (my run, 02:24Z)	8/8 and 8/8
Python mutation, dt > gap_ms → >=	fails 3 of 14 (covered_ms, gaps_over_20s, covered_pct), as the README says. The Kotlin mutation (2 of 8) is Opus's figure; not re-run
Section 9 rows from the 9 exports in docs/roundDownloads/	all 7 rows of the handoff table reproduced (FT3 143.9 min / 7,858 / 73.0% / 16 / 11.2 min / 3.1 m through FT7 141.2 / 4,585 / 43.9% / 6 / 65.6 / 3.1)

3. Verdict (question 6)

PASS: both APKs may go on his phone for the carries and on the course. What it rests on:

1. **Data integrity (question 5):** no path loses or corrupts a row the recorder handed over; the worst case is bounded and labelled (Section 8).
2. **K with the screen locked (question 1):** the configuration is the canonical one and nothing in the code drops fixes; the one path untested on the S26 is a restart after Android kills the process (Section 4).
3. **T raw and fair (question 2):** in the round, T's fixes are K's fixes, byte for byte, minus one swapped at a config re-apply (Section 5).
4. **The measure (question 3):** the phone and the PC agree with each other and with Section 9, on the right time base (Section 6).
5. **The heartbeat (question 4):** it can run late only when the CPU sleeps, which cannot happen while K runs; the tool's wording for that case should change before a T-alone carry, but no verdict depends on it (Section 7).
6. **The round (Section 9):** 0 gaps, longest interval 1.9 s (K) and 1.6 s (T), through five golf-tracker gaps totalling 782 s and one 376 s screen-off.

Not part of this PASS: the bar itself. His bar names a locked screen and music; that carry has not happened. This verdict says the apps are fit to run it, not that either has won.

Conditions attached (none blocks the carry):

- Reinstall bakeoff-2 with `adb install -r` as setup does (same application ids, sessions and grants kept) and read every grant back, as planned in NEXT_CHAT_2026-09-14.md item 5.
- Never uninstall either app while an export it made is still on the phone and unpulled (Section 8, the export trap).
- Before a T-alone carry, two small changes are recommended for Opus (Sections 7 and 9): the tool's wording when heartbeats are absent, and light Doze in DeviceState. Neither changes a recorder.

4. K with the screen locked on a Galaxy S26 (question 1)

Read in RecorderService.kt / Recorder.kt (handwritten flavor) [measured]:

```
LocationRequest.Builder(PRIORITY_HIGH_ACCURACY, 1000), setMinUpdateIntervalMillis(1000),
setMinUpdateDistanceMeters(0f), setMaxUpdateDelayMillis(0), GRANULARITY_FINE,
setWaitForAccurateLocation(false); delivered on its own HandlerThread;
ServiceCompat.startForeground(..., FOREGROUND_SERVICE_TYPE_LOCATION); START_STICKY; a
non-reference-counted PARTIAL_WAKE_LOCK acquired with the request and released with it;
Recorder.onProcessStart restarts the service when a session is active and the service is not; manifest declares
FOREGROUND_SERVICE_LOCATION, ACCESS_BACKGROUND_LOCATION, WAKE_LOCK,
foregroundServiceType="location". setup grants fine, coarse, background location and notifications, puts the app
```

on the Doze whitelist (dumpsys deviceidle whitelist +), allows RUN_ANY_IN_BACKGROUND, and reads each back (bakeoff.ps1 Grant-All).

What could lose fixes with the screen locked, in order of likelihood:

1. **The process is killed and the restart is refused.** Android 12+ refuses startForeground from the background unless exempt. The exemption K relies on is on Android's list, verbatim: "The user turns off battery optimizations for your app." [design, fetched]. Both apps are Unrestricted (unrestricted=1 at all 1,445 K beats and 1,447 T beats; standby bucket 5 = STANDBY_BUCKET_EXEMPTED at every beat [measured]). If a restart were refused, K logs foreground_start_failed / service_start_failed, keeps the session active, and retries at the next process start; the gap stays in the file. Seen working on the emulator, Android 15, n = 2 kills (back in about 1 s and 1.3 s). **Not yet seen on the S26: n = 0 restarts** (no previous_exit in either round log).
2. **Samsung's app sleeping.** Unrestricted and "Never auto sleeping apps" exclude each other on One UI 8.5 (measured by Opus 2026-09-13); that Unrestricted covers sleeping is [inferred] from that exclusivity, not documented. Two hours with 5.3% screen-off is not a test of it. The carries are.
3. **Nothing in the code.** The callback cannot throw on a fix (toFix returns null for a fix without a time and the event is logged); a write failure is counted (write_failures=0 at every beat); onTaskRemoved only logs; the notification is IMPORTANCE_LOW and ongoing, and dismissing a foreground notification on Android 14+ does not stop the service [design].
4. **Doze.** Deep Doze needs a stationary phone; a walking carry does not enter it, and a whitelisted app keeps its wake lock in Doze anyway [design]. Light Doze is not visible in the logs: DeviceState reads isDeviceIdleMode(), which is deep idle only; isDeviceLightIdleMode() (API 33+) is not read [measured in DeviceState.kt:29; API level: verify]. Recommended for Opus before the carries: add it to the snapshot as light_doze.

What the round shows [measured, n = 1 round]: updates_on 41 ms after START; availability available=false then true within 16 ms of it (the provider's first answer, not a loss); first fix 0.166 s after START at 92.9 m (the provider's warm-up fix; every later fix is 10.2 m or better, n = 7,236); last fix 0.785 s before STOP; 7,236 intervals of which 7,234 round to 1 s and 2 to 2 s, longest 1,884 ms; delivery latency (wall clock minus fix time) median 19 ms, p90 26 ms; median accuracy 3.06 m, p90 3.79 m; 0 repeated fix times; write_failures 0; heartbeats every 5.0 s (median 5,008 ms, max 6,587 ms, one beat over 6 s); thermal 0, battery saver 0, charging 0.

5. T: raw and fair? (question 2)

The four changed defaults are set as the README says [measured, transistor/.../Recorder.kt:174-196]: desiredAccuracy HIGH, distanceFilter 0, locationUpdateInterval 1000, fastestLocationUpdateInterval 1000, disableElasticity true, allowIdenticalLocations true, filter.policy PassThrough, filter.useKalman false, filter.trackingAccuracyThreshold 0.0, activity.disableStopDetection true; stopOnTerminate false, enableHeadless true, startOnBoot false, no url (so no uploads), maxDaysToPersist 30. changePace(true) follows start(), and on a process restart only if the SDK did not come back moving.

Is anything else in the SDK altering or dropping fixes? Read from the 4.5.1 binary [measured]:

- `TSLocationManager.onLocationResult` runs every incoming `LocationResult` through `LocationMetricsEngine.computeFor` and `LocationFilter.evaluate`, and a REJECTED decision drops the fix ("Rejected location:" is logged). The filter's reason strings are low-accuracy, stationary-drift, implied-speed, outlier-capped, kinematic-cap, distance-filter-cap. That is more than the README's list (accuracy gate, implied speed, outliers). low-accuracy is gated by the threshold, which is 0. Which of the others survive PassThrough I could not settle from obfuscated bytecode alone, so the field data decides:
- **In the common window (16:07:24 to 18:07:54) K delivered 7,230 distinct fix times and T 7,230; 7,229 are shared, and every shared fix has byte-identical latitude, longitude, accuracy and elapsed_rt_ms in both logs (7,230 of 7,230 across the whole overlap).** The SDK altered no coordinate and no accuracy. One fix differs: K has 16:07:34.967, T instead has 16:07:34.398 three times, 0.3 s after `sdk_ready_on_resume` at 16:07:34.695. That is the config re-apply re-requesting updates from the provider, which answers with its last fix before the stream resumes. It cost T one fix and made no gap (T's longest interval 1,568 ms). Everything else the provider produced reached T's log. [measured, n = 7,230]

- T's moving flag was 1 on all 7,256 rows and the store's `is_moving` true on all 7,253; sample was 0 on all rows; event was empty except two `motionchange` rows. No sampling burst, no stationary state, no geofence.
- The SDK acquires no wake lock: across its 642 classes, `android/os/PowerManager` is referenced only by `DeviceSettings` (the screen that opens system battery settings). Its services are declared `foregroundServiceType="location"`. So T's fixes arrive on the provider's own wake-ups, and while K runs beside it, on K's wake lock. **This is the one fairness gap in the design: T has never run without K's wake lock.** Hence the T-alone carry (Section 9).
- The web docs' descriptions of `distanceFilter` (default 10 m), `locationUpdateInterval` (1000 ms), `fastestLocationUpdateInterval` ("-1, not set"), `disableElasticity` (false) and `deferTime` (0) were fetched and match the settings above [design]. `speedJumpFilter` appears in the binary only as a `TSConfig` key; the teleport check it fed (`TSLocationManager.d: "Detected invalid location (teleport)"`) now reads `LocationFilterState.maxImpliedSpeed`, part of the filter above.

Does `ready()` on every process start and `changePace(true)` keep it recording through a five-minute wait on a tee? In the round, the two longest stays within 10 m of one point were **414 s** (16:07:18 to 16:14:11) and **393 s** (16:35:55 to 16:42:28); T kept 415 and 394 fixes through them, longest interval 1,568 ms and 1,097 ms, `moving=1` throughout [measured, $n = 2$ stays over 5 min]. So yes, on this evidence. `ready()` re-applies the config on every `onResume`; each re-apply costs the fix swap above and three duplicate rows (Section 11), never a gap ($n = 3$ resumes).

T's round numbers [measured]: first fix 0.150 s *before* START (the SDK's `start()` emits its last known location, stamped before our START), last fix 1.144 s before STOP; 7,255 intervals: 7,242 round to 1 s, 12 to 0 s (the duplicates), 1 to 2 s, longest 1,568 ms; latency median 26 ms, p90 34 ms; the store's `recorded_at` minus timestamp median 25 ms; median accuracy 3.06 m, p90 3.79 m, max 10.2 m; heartbeats max spacing 5,037 ms.

6. The measure (question 3)

`Coverage.of` (Kotlin) and `coverage()` (Python) compute the same thing: sort fix times; an interval over 20,000 ms is a gap, otherwise it is covered; longest = max interval; span = last minus first; median accuracy over fixes that carry one; pass = at least 2 fixes, 0 gaps, covered 99% or more. Both readers accept a row only with exactly the header's column count and a whole number in `fix_ms`, count the rest as skipped, and treat an empty `acc_m` as absent [measured, both files read line by line]. Two reader differences exist and neither can bite: Python uses the `csv` module and Kotlin `split(' ', '')`, but `Csv.token` strips commas from the only free-text cells and numbers carry none; Kotlin has no edge gaps, which the README states (the phone's number is a glance; the PC gives the verdict).

The choices asked about:

- **Fix time, not receive time.** Right. Section 9 was computed on the web app's `ts`, which is the position's own timestamp; `fix_ms` is `Location.getTime()`, the same thing. Receive time would hide a recorder that delivers late and forgive one that batches. Receive time is kept in `wall_ms` beside it (latency above).
- **T's samples counted.** Right for the app log, and harmless this round (0 samples). A sample is a fix the SDK hands the app but does not persist, so counting them can only make the app log look better than the store; the store is scored separately, which closes that door.
- **Cut rows skipped.** Right, and it cannot yield a wrong timestamp: `fix_ms` is column 2 of 15, so a row cut anywhere before its 15th column has too few columns and is skipped; a row cut inside the 15th column (event, T only) keeps a complete timestamp and loses only part of a token [measured by reading the writer and both readers].

One thing the tool should say and does not yet (Section 7): a beat that is late is not a beat that is missing.

7. The gap diagnosis: can the heartbeat call a live recorder dead? (question 4)

The ticker is `Handler.postDelayed` on a `HandlerThread` (`SessionLog.kt:159-171`). `Handler` timing is `SystemClock.uptimeMillis`, which stops while the CPU sleeps [design]. So a 5 s beat can arrive late by however long the CPU slept. Each event row also carries `elapsed_rt_ms` (`SystemClock.elapsedRealtime`, which keeps counting in sleep), so a late beat is detectable after the fact.

- **K: no.** K holds a partial wake lock from `updates_on` to `recorder_halt`; the CPU does not sleep while K's recorder is alive. If the service dies and the process lives, the wake lock goes with it, the beats continue without it, and they say `recorder=0`, which is true.

- **T beside K: no**, for the same wake lock (it is the phone's CPU, not K's).
- **T alone: yes, the wording can be wrong.** With no wake lock, a live SDK with a quiet GPS (trees, a building, a long stand indoors) leaves nothing to wake the CPU; the beats stall; `diagnose_gap` then prints "no heartbeat at all: the app was not running", which would be false. Fixes keep waking the CPU while they arrive, so this cannot mislabel a gap that has fixes in it; it can only mislabel a gap with none, which is exactly when the label matters.

In the round [measured]: K's 1,444 beat intervals, median 5,008 ms, max 6,587 ms (one over 6 s, at 16:07:21, during START's file opens); T's max 5,037 ms. The one recorder=0 beat in each log is the first beat, 17 ms after START, before `updates_on / sdk_started` (`since_fix_ms=-1`, `fixes_this_process=0`). No stall, as expected with the screen on 95% and K's wake lock held throughout.

Recommended for Opus, before a T-alone carry (`tools/track-coverage.py`, `diagnose_gap`): (a) never print "the app was not running" from heartbeat absence alone; print "no heartbeat in the gap" and let `previous_exit`, `log_open` and `sticky_restart/sdk_restored` within the gap say whether the process died; (b) when beats exist but are sparse, report their spacing in `elapsed_rt_ms` so a sleeping CPU reads as "beats late by up to N s", not as a dead recorder. The rows already carry what this needs. I did not change the tool in this run (Section 14, decision 2).

8. Data integrity (question 5)

Read: `SessionLog.kt`, `Sessions.kt`, `FixCsv.kt`, `LocationFix.kt`, `Exporter.kt`, `BakeoffApp.kt`, `MainActivity.stopRecording`, both `Recorder.stop`.

- **Flush per row:** `line()` writes and flushes to the OS per row under the object lock; `fix()` and `event()` are `@Synchronized`, so K's callback thread, T's main-thread listener and T's EventBus headless thread cannot interleave a row. A killed process loses nothing flushed [design].
- **fsync every 15 s:** bounds what a power loss can take to 15 s of rows, plus `close()` syncs at STOP. Documented, and the right trade.
- **Cut-row repair:** `open()` reads the last byte; if it is not a newline it writes one before appending; the readers skip the short row (Section 6). Sound.
- **A row is never invented:** `toFix` returns null when `Location.time <= 0` and the event log records `fix_without_time`; absent fields are empty cells (`FixCsv.csvRow`). Nothing in a row is inferred.
- **The active session survives a death:** `Sessions.begin` commits the id synchronously before the recorder starts; `BakeoffApp.onCreate` reopens the log and logs `previous_exit` from Android's own record.
- **STOP order:** K halts synchronously before `stop` is written; T's `BGGeo.stop()` is launched on Main and runs after `log_close`, so a fix the SDK emits in that instant is dropped from the log by design (the log is closed) and stays in the SDK store outside the window. In the round the store has 0 rows after STOP.

What can lose data, and how much:

1. **meta.json is not fsynced** (`Sessions.begin`, `writeText`). A power loss in the seconds after START could leave it empty; the tool then has no `started_wall_ms`, prints the table without the session line and cannot see a leading edge gap. Rows are untouched. Minor; worth an fsync when the recorder is rebuilt.
2. **The export trap.** `Exporter.copy` deletes this app's earlier MediaStore entry by path and name, then inserts. Android lets an app delete only rows it owns; after an **uninstall and reinstall** the old files are orphans, the delete silently fails, MediaStore writes `fixes (1).csv`, and `bakeoff.ps1` pull scores the stale `fixes.csv` from the old install as if it were the new session. An in-place install `-r` (what setup does and what the rename needs) keeps ownership and does not trip this. Rule for the test week: never uninstall with unpulled exports on the phone; if it ever happens, clear `Download/golf-bakeoff` over adb before the next EXPORT.
3. `Exporter.copy` **returns 0 silently** when `insert` or `openOutputStream` returns null; the toast's file count is the only sign. The app's private copy is untouched, so nothing is lost, only not exported.

K's second export at 20:03: consistent with the code. EXPORT ALL copies every session again and replaces the earlier copies; the session had closed at 18:07:54, so the content is identical and only MediaStore's timestamps moved. His "3 but 1 probably" is the likely reading; the data cannot distinguish a re-export from a first export at 20:03 [inferred].

Round: rows skipped: 0 in both logs, `write_failures` 0 at every beat.

9. The round: does it count, and what a fair locked-screen test needs

Reproduced: Section 2, all three outputs identical.

Does it count toward the verdict?

- **As the on-course test: yes.** Real sky at Veenker, pocket and push cart ("always near the ball"), 2.0 hours, both apps beside golf-tracker v23 on the same phone; 0 gaps; median accuracy 3.06 m in both (n = 7,237 / 7,256), equal to the field-validated web number (3.0 to 3.2 m, FT3 to FT6). Both kept 235 / 21 / 76 / 383 / 64 fixes through golf-tracker's five gaps.
- **As the bar's locked-screen clause: no.** Screen off at 76 of 1,445 beats (5.3%), music at 0 of 1,445. Beyond the numbers, the round was a kind environment: golf-tracker kept the screen on and held its own high-accuracy location request for 95% of the time, so the GNSS engine and the CPU were hot for every app on the phone, and K's wake lock covered T.
- **What it does show about locked screens:** in the one screen-off stretch, 17:41:18.484 to 17:47:34.670 (376 s), K and T each logged 376 fixes, longest interval 1,205 ms, with 75 beats reading screen_on=0, while golf-tracker logged 0 fixes from 17:41:17 to 17:47:41 [measured, n = 1 stretch]. That is the first evidence on the S26 that a native foreground service holds GPS through a locked screen where Chrome cannot. One stretch is one stretch.

Why the four golf-tracker gaps with no screen event stay unexplained: the test apps' logs record the phone's screen and Doze, not Chrome's tab state. A tab switch, a music change or Chrome throttling the page leaves no trace in these logs. The handoff already says the web app cannot hold the receiver from the background; these gaps are consistent with that and prove nothing more.

A fair locked-screen test (my decision, Section 14, decision 1; he can change any number):

Condition	Rule	How it is checked
Length	2.0 h or more per carry, START to STOP	session: line of the tool
Where	Walking outdoors for most of it, phone locked in a pocket, unplugged	charging=0 at every beat; the track itself
Screen	Locked; screen_on=0 at 90% or more of heartbeats	count screen_on=0 beats in events.csv
Music	A music app playing through headphones; music=1 at 50% or more of heartbeats	count music=1 beats
Other apps	golf-tracker closed (the Chrome tab, not just backgrounded); nothing else requesting location	his word; Chrome's own track absent
The apps	Carry 1: both, started together. Carry 2: T alone (K not started)	bakeoff.ps1 start starts both; for carry 2 Opus starts T only
Hands off	Do not open either test app until STOP; opening T re-applies ready() (three duplicate rows each time, no gap)	sdk_ready_on_resume count
Scoring	The bar per app, START to STOP with edges: 0 gaps > 20 s (99% follows). Report the screen and music shares beside it so the clause is proven, not assumed	tools/track-coverage.py

Why T alone: Section 5, the wake lock. Why not K alone: K carries its own wake lock and foreground service, and carry 1 already tests K in company that adds nothing to it. Why golf-tracker must be closed: a second high-accuracy request from Chrome keeps the receiver hot on behalf of both recorders and keeps the screen on; the round just showed how much that masks. A walking round without golf-tracker would be an equally fair carry 1 at the cost of a round's logging; his call.

Before the carries, recommended for Opus: light Doze in the snapshot (Section 4, item 4) and the tool wording (Section 7). Both are shared code and the tool, not a recorder; neither needs xhigh.

10. Classify only: what carries into the recorder that feeds rounds (question 7)

Not designed here. Classification of the code at 645355a:

Reusable as it stands (behaviour, and mostly the code):

- LocationFix.toFix + Fix + Csv: one path from an Android Location to a row, every cell measured, absent = empty, no time = no row. This is the "measured and inferred are never mixed" rule already in code.

- SessionLog's write posture: append-only, flush per row, fsync cadence, cut-row repair, a heartbeat with the phone's state, and the process-exit record from `BakeoffApp.logExitsSinceLastLook`. Add the meta fsync.
- K's RecorderService shape: location foreground service, `START_STICKY`, partial wake lock, resume on process start from a committed "active" flag.
- Coverage: the Section 9 measure, held to the shared fixture with the PC tool.
- T's two integration rules (`BackgroundGeolocationHeadlessTask`, `ready()` on resume) if T is the winner, plus the configuration block.

Must be rebuilt under the data model rules (xhigh on his word):

- **The session model.** A bake-off session is START to STOP of an app; a round's track belongs to a round record with an id, a course, holes and marks, and today lives in `trackstore.js` as `[lat, lon, acc, ts]` per fix and in the export as `tracks: { roundId: [...] }`. Attaching a native track to a round, and the fields a native fix adds (`elapsed_rt_ms`, `speed`, `bearing`, `provider`, `mock`), is a schema change.
- **Storage and export.** CSV files in private storage plus a MediaStore copy are a test rig, not the round store. The round store needs a version and a `migrate()` step, and the logged rounds (six field tests plus this one, in `docs/roundDownloads/` and this folder) must come across without a value changing meaning. That is a migration of logged real data.
- **T's SDK store is never the source of truth.** Its rows carry SDK-derived fields (`is_moving`, `activity`, `odometer`, `event`) beside the measured fix, and it persisted the same fix under four uuids at START (Section 11). A production app on T keeps writing its own log through `recordSdkLocation`, deduplicated on (`fix_ms`, `elapsed_rt_ms`), and treats the SDK store as scratch. Same rule as quarantine: the SDK's inferences travel labelled or not at all.
- **The GPS pipeline itself.** Whichever recorder wins replaces the role of `js/gps/gps.js`; that is a GPS precision pipeline item by his Section 4 rule, xhigh on his word, and the geodesy stays `js/util/geo.js`'s.

Throwaway: `MainActivity`, `Exporter`, the two-flavor split, `bakeoff.ps1` beyond its adb recipes.

So: turning the winner into the round recorder is two of his four xhigh categories at once (GPS precision pipeline; data model and the migration of logged rounds). Nothing of it is designed in this report.

11. Repeated fix times (question 8)

Where they come from, in the round [measured, $n = 1$ round]: T's log has 10 extra rows at 4 fix times; every extra row is byte-identical to the row it repeats (latitude, longitude, accuracy, `elapsed_rt_ms`) and to K's fix at that time, and every one sits within 1 s of an SDK state event:

Fix time	Rows (log)	Rows (store)	Event within 1 s
16:07:23.967	5 (1 motionchange + 4)	4, all before START	<code>sdk_started</code> , <code>sdk_motionchange</code> , <code>sdk_ready_on_resume</code>
16:07:34.398	3	2	<code>sdk_ready_on_resume</code> (16:07:34.695)
16:15:11.965	3 (1 motionchange + 2)	3	<code>sdk_motionchange</code> <code>moving=true</code> (16:15:12.804), the SDK's own, while already moving
18:08:07.017	3	2	<code>sdk_ready_on_resume</code> (18:08:07.415), opening the app to stop

So a repeat is **the SDK re-emitting the provider's last fix on its own state transitions** (`start`, `changePace`, a config re-apply, a motion change), not the provider delivering twice and not a new fix at old coordinates. The store persists each re-emission as a new record (distinct uuids, 0 duplicate uuids). After a `kill -9` the restart path runs the same transitions (`ready()`, `restore`, possibly `changePace`), which is why runs 2 and 3 saw them without our call; the emulator's 88 in run 2 is not explainable from the field data and stays Opus's observation [verify].

Configuration or analysis? Analysis. Nothing to change in T's config: the repeats carry no false position and cost no coverage, and the SDK offers no switch for its own re-emissions. For the tool: count "SDK re-emissions" (same `fix_ms`, `elapsed_rt_ms`, `lat` and `lon`) separately from "distinct fixes at one time" (0 seen), so the fix count is honest without a footnote. For the production recorder on T: dedupe at write time on (`fix_ms`, `elapsed_rt_ms`) (Section 10).

12. `allowIdenticalLocations` (question 9)

The doc, fetched verbatim: "By default, the SDK ignores a location that is identical to the previous one. Set true to record every location regardless of duplication." What "identical" means is in the binary [measured, 4.5.1]:

- `CLLocationManager.e(Location)` compares the new fix with the last one and returns identical when **time, latitude and longitude are equal**, OR when **latitude, longitude, speed and bearing are equal regardless of time**. The second branch would call a new fix at the same spot identical: in K's raw stream that happened on 149 of 7,236 consecutive pairs (2.06%), in 124 runs of up to 4 fixes, and 209 pairs (2.89%) repeat latitude and longitude alone [measured]. Those are stands, which is the shot.
- **But nothing calls it.** No class in the AAR invokes `e(Location)`, and no class reads `GeoState.allowIdenticalLocations` except the Kotlin config wrapper that sets and gets it (a string search over all 642 classes, then `javap -c` of every class that references `CLLocationManager`). The flag is stored, edited and serialised, and consulted by nothing in the tracking path. The round agrees: with true, the SDK persisted the same fix under four uuids at START, which a live identical-filter would have dropped and a dead one lets through either way.

So: in 4.5.1 the setting is inert; false would be neither cleaner nor riskier today. It stays true (Section 14, decision 3), because the comparison that exists, if a later SDK wires it back in, drops real fixes at stands 2% of the time, and a re-emission is a row the tool can count while a dropped stand is a hole nothing can recover.

13. Restart backoff (question 10)

Android schedules the restart of a killed service with a delay that grows when the service dies again within its reset window; that is the `ActiveServices` restart logic in the system server, not anything an app configures [design; the constants were not fetched, the AOSP page truncated, so no numbers are quoted here]. The two emulator measurements (about 5 s after one kill, about 57 s after a second kill minutes later, $n = 2$) are the shape of that logic. Neither recorder can shorten it.

What a recorder can do, and only the second is a design item:

1. **Not die.** Unrestricted battery, a location foreground service, a small process. Both apps have all three.
2. **Not wait for the system.** An independent wake-up (an exact alarm or `WorkManager` job that restarts the foreground service when the recorder is found dead) is allowed from the background under the same exemption as the sticky restart ("The user turns off battery optimizations for your app.") [design, fetched]. The SDK carries its own `SCHEDULE_EXACT_ALARM` and `RECEIVE_BOOT_COMPLETED` permissions for this kind of thing (its manifest); K has nothing of the kind. That is for the production recorder (Section 10), not for the bake-off.

For the field: watch, do not change. A natural death shows as `previous_exit` with Android's reason, followed by `sticky_restart / updates_on (K)` or `sdk_restored (T)`; the gap between the exit time and the restart event is the backoff. `kill -9` from adb is not what the field does; the field's killers are memory and Samsung sleeping, and both leave that record.

14. Decisions made alone (logged in docs/DECISIONS_LOG.md)

1. **The round counts as the on-course pass and not as the bar's locked-screen clause, and the fair test is defined by measurable thresholds** (Section 9: 2.0 h or more, `screen_on=0` at 90% or more of beats, `music=1` at 50% or more, golf-tracker closed, both together then T alone). Reason: the bar is his words and names the screen and music; the heartbeats can prove both; the wake-lock finding makes a T-alone run the only fair reading of T. The numbers are mine; he can move them.
2. **No code changed in this run.** The four recommendations (tool wording for absent heartbeats; light Doze in `DeviceState`; SDK re-emissions counted apart from repeats; `fsync` of `meta.json`) do not bear on the verdict, three are Opus's files, and his usage cap on this project says spend the run on the answers. They are listed for Opus with file and function names.
3. `allowIdenticalLocations` **stays** true. Reason: Section 12.

15. What Matt must decide

- Nothing to run this review. Two things for the week: whether carry 1 is a desk-started walk or a walking round without golf-tracker (Section 9), and whether Opus makes the four small changes before the carries. Both can be

one-line answers to Opus.

16. Answered line

2026-09-14 · 2.3 · PASS: both APKs at 645355a may go on his phone and on the course; no path loses or corrupts a row, K's config is canonical, T's fixes are K's byte for byte (7,230 of 7,230 shared) with the SDK altering nothing; the measure matches Section 9 (7 of 7 rows reproduced). The 2026-09-14 round counts as the on-course test (0 gaps, longest 1.9 s / 1.6 s, 3.06 m median, n = 1 round) and NOT as the bar's locked-screen clause (screen off 5.3% of beats, music 0%); one 376 s screen-off stretch held 1 Hz in both. A fair locked-screen test: 2 h or more walking, screen off at 90% or more of beats, music at 50% or more, golf-tracker closed, both apps together and then T alone, because K's wake lock keeps the CPU on for T and the SDK holds none of its own. Found in the SDK binary: allowIdenticalLocations is inert in 4.5.1 (no caller), so it stays true. Repeated fix times are SDK re-emissions on its own state events (10 rows at 4 times, all byte-identical to K's fix); analysis, not configuration. Nothing changed in code; four small changes recommended for Opus before a T-alone carry. docs/handoff/REPORT_2.3.md.